NETAJI SUBHAS INSTITUTE OF TECHNOLOGY, BIHTA

Approved by AICTE: Affiliated to B.E.U. & S.B.T.E, Bihar

Board Roll No.....

Name : Suraj Arya

Session : 2024-28

Year/Semester: 3rd Sem / 2nd Year Roll No. 241064

Branch : Computer Science Engineering

Subject : Data Structure & Algorithm

EXTERNAL LAB RECORD

INDEX

Lab Name: Data Structure & Algo. Lab Faculty Incharge : R.K. Singh Sir

Lab Incharge: Bipin Sir.

Sr. No.	Experiments Name	Date of Experiment	Completion Date	Page No.	Signature	Remarks
1	Linear Search in C	05/02/26	05/02/26	1-2		
2	Binary Search in C	19/02/26	19/02/26	3-5		
3	Quick Sort in C	26/02/26	26/02/26	6-8		
4	Merge Sort in C	19/03/26	19/03/26	9-11		
5	Bubble Sort in C	26/03/26	26/03/26	12-14		
6						
7						
8						
9						
10						

Date: 05/02/26

Page No.: 1

Experiment:-1

Linear Search in C.

=> Aim:

To write a C program to perform Linear Search on an array.

=> Theory:

Linear Search is a Simple Searching technique in which each element of the array is checked Sequentially until the desired element is found or the list ends.

* It is easy to implement
* Suitable for small datasets
* Time Complexity: O(n)

=> Algorithm:

1. Start
2. Input number of elements (n)
3. Enter array elements
4. Enter input element to Search (key)
5. Traverse array from 0 to n-1.
6. If element equals key => display position
7. If not found => display "Element not found"
8. Stop

=> Program (C code):

```
#include <stdio.h>
int main() {
    int arr[100], n, i, key, found = 0;
```

Output:

```
Enter number of elements: 5
Enter elements:
10 20 30 40 50
Enter element to search: 30
Element found at position: 3.
```

```
printf("Enter number of elements: ");
scanf("%d", &n);

printf("Enter elements: \n");
for(i=0; i<n; i++){
    scanf("%d", &arr[i]);
}

printf("Enter element to search: ");
scanf("%d", &key);

for(i=0; i<n; i++){
    if(arr[i] == key){
        printf("Element found at position %d", i+1);
        found=1;
        break;
    }
}
if(found==0){
    printf("Element not found");
}
return 0;
}
```

Result:

The program for Linear search was successfully executed and the element was found at the desired position.

Date: 19/01/26 Experiment No: 2. Page No.: 3

Binary Search in C.

Aim:

To write a C program to perform Binary search on a sorted array.

Theory:

Binary search is an efficient searching technique that works on sorted arrays. it repeatedly divides the search interval into half.

* compare key with middle element.
* if equal → found.
* if smaller → Search left half.
* if larger → Search right half.

Time Complexity O(log n)

Algorithm:-

1. Start.
2. input number of elements (n)
3. Enter sorted array elements
4. Input element to search (key)
5. Repeat while low <= high.
   * Find mid = (low + high) / 2
   * if arr[mid] == key → found.
   * if key < arr[mid] → high = mid - 1
   * else → low = mid + 1

7. if not found => display message.
8. Stop.

Program (C code):

```
#include <stdio.h>
int main () {
    int arr[100], n, i, key;
    int low, high, mid, found=0;

    Printf (" Enter number of elements. ");
    Scanf ("%d", &n);

    printf (" Enter sorted elements \n");
    for (i= 0; i<n; i++){
        scanf ("%d", &arr[i]);
    }

    Printf (" Enter element to Search: ");
    Scanf ("%d", &key);

    low = 0;
    high = n-1;

    While (low <= high) {
        mid = (low + high) / 2;
        if (arr[mid] == key) {
            Printf (" Element found at position %d", mid+1);
            found = 1;
```

Output

Enter number of elements: 5
Enter sorted elements:
10 20 30 40 50
Enter element to search 40
Element found at position 4.

```
        break;
    }
    else if (key < arr[mid]) {
        high = mid + 1;
    }
    else {
        low = mid + 1;
    }
}

if (found == 0) {
    printf("Element not found");
}
return 0;
}
```

Result:- The program for binary search was successfully executed and the element was found using divide- and-conquer technique.

Date: 26/02/26

Page No.: 6

Experiment No. 3.

Quick Sort in C.

Aim:

To write a C program to perform Quick Sort on an array.

Theory:-

Quick Sort is a divide and conquer sorting algorithm. It selects a pivot element and partitions the array such that:

* Elements smaller than pivot => left side.
* Elements greater than pivot => right side.

Then recursively sorts the subarrays.

Time Complexity:

* Best / Average: $O(n \log n)$
* Worst: $O(n^2)$

Algorithm:

1. Start.
2. Input number of elements (n)
3. Enter array elements.
4. Choose last element as pivot.
5. Partition array into two parts.
6. Recursively apply Quick Sort on left and right subarrays.
7. Display sorted array
8. Stop.

Program (C code):

```
#include <stdio.h>

//Function to Swap elements
void Swap (int *a, int *b) {
    int temp = *a;
    *a = *b;
    *b = temp;
}

//Partition function
int partition (int arr[], int low, int high) {
    int pivot = arr[high];
    int i = low-1;

    for (int j = low; j<high; j++) {
        if (arr[j] < pivot) {
            i++;
            Swap (&arr[i], &arr[j]);
        }
    }

    Swap (&arr[i+1], &arr[high]);
    return i+1;
}

// Quick Sort function
void quickSort (int arr[], int low, int high) {
    if (low<high) {
        int pi = partition (arr, low, high);
```

Output

```
Enter number of elements: 5
Enter elements:
50 20 40 10 30

Sorted array:
10 20 30 40 50
```

```
    quickSort(arr, low, pi-1);
    quickSort(arr, pi+1, high);
}

int main(){
    int arr[100], n, i;
    printf("Enter number of elements: ");
    scanf("%d", &n);
    printf("Enter elements:\n");
    for(i=0; i<n; i++){
        scanf("%d", &arr[i]);
    }
    QuickSort(arr, 0, n-1);
    printf("Sorted arr:\n");
    for(i=0; i<n; i++){
        printf("%d", arr[i]);
    }
    return 0;
}
```

Result:- The program for Quick Sort was successfully executed and the array was sorted using divide-and-conquer technique.

Date: 19/03/26 Experiment No. 4 Page No.: 9

Merge Sort in C.

Aim:-

To write a C program to perform Merge sort on an array.

Theory:-

Merge sort is a divide-and-conquer sorting algorithm. It divides the array into two halves, recursively sorts them, and then merges the sorted halves.

* Stable sorting algorithm
* Works well for large datasets.

Time Complexity: $O(n \log n)$.

Algorithm:-

1. Start
2. Input number of elements (n)
3. Enter array elements
4. Divide array into two halves
5. Recursively sort both halves
6. Merge the sorted halves
7. Display sorted array
8. Stop

Program (C Code):

```
#include <stdio.h>
```

```
// function to merge two sub arrays.
void merge (int arr[], int low, int mid, int high) {
    int i = low, j = mid + 1, k = low;
    int temp[100];

    while (i <= mid && j <= high) {
        if (arr[i] < arr[j]) {
            temp[k++] = arr[i++];
        } else {
            temp[k++] = arr[j++];
        }
    }
    while (i <= mid) {
        temp[k++] = arr[i++];
    }

    while (j <= high) {
        temp[k++] = arr[j++];
    }

    for (i = low; i <= high; i++) {
        arr[i] = temp[i];
    }
}

// Merge Sort function
void mergeSort (int arr[], int low, int high) {
    if (low < high) {
        int mid = (low + high) / 2;
```

Output:

Enter number of elements: 5

Enter elements:

38 27 43 3 9

Sorted array:

3 9 27 38 43

```
    mergeSort(arr, low, mid);
    mergeSort(arr, mid+1, high);
    merge(arr, low, mid, high);
  }
}

int main() {
  int arr[100], n, i;

  printf("Enter number of elements: ");
  scanf("%d", &n);

  printf("Enter elements:\n");
  for(i=0; i<n; i++) {
    scanf("%d", &arr[i]);
  }

  mergeSort(arr, 0, n-1);
  printf("Sorted array:\n");
  for(i=0; i<n; i++) {
    printf("%d", arr[i]);
  }
  return 0;
}
```

Result:

The program for merge sort was successfully executed and the array was sorted using DAC.

Date: 19/03/24 Experiment No. 5. Page No.: 12

Bubble Sort in C.

Aim:

To write a C program to perform Bubble Sort on an array.

Theory:

Bubble Sort is a simple sorting algorithm that repeatedly compares adjacent elements and swaps them if they are in the wrong order.

* Largest element "bubbles" to the end in each pass.
* Easy to understand and implement.

Time Complexity:

* Best Case: $O(n)$
* Worst / Average: $O(n^2)$

Algorithm:

1. Start
2. Input number of elements (n)
3. Enter array elements.
4. Repeat for $(n-1)$ passes.
   * Compare adjacent elements.
   * Swap if left element > right element.
5. Display sorted array.
6. Stop

```
return 0;
}
```

Result:-

The Program for Bubble Sort way Successfully executed and the array was Sorted Correctly.

Output:

Enter number of elements: 5
Enter elements:
64 34 25 12 22
Sorted array:
12 22 25 34 64

Program (C code):

```
#include <stdio.h>
int main(){
    int arr[100], n, i, j, temp;

    printf("Enter number of elements: ");
    scanf("%d", &n);

    printf("Enter elements: \n");
    for(i=0; i<n; i++){
        scanf("%d", &arr[i]);
    }

// Bubble Sort.
for(i=0; i<n-1; i++){
    for(j=0; j<n-i-1; j++){
        if(arr[j] > arr[j+1]){
            temp = arr[j];
            arr[j] = arr[j+1];
            arr[j+1] = temp;
        }
    }
}

printf("Sorted array: \n");
    for(i=0; i<n; i++){
        printf("%d", arr[i]);
    }
```